QWEN.md — HelixQA VisionEngine

Field	Value
Revision	1
Created	2026-05-23
Last modified	2026-05-23
Status	active
Status summary	Created per Phase 39.IT (User mandate 2026-05-23) — propagation of QWEN.md across the consumer fleet, mirroring CLAUDE.md + AGENTS.md per §11.4.35 canonical-root inheritance.
Issues	none
Continuation	—

INHERITED FROM constitution/QWEN.md

All rules in `constitution/QWEN.md` (and the `constitution/Constitution.md` it references) apply unconditionally. This module's rules below extend them — they do NOT weaken any universal clause. When this file disagrees with the constitution submodule, the constitution wins. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

The universal anti-bluff covenant (§11.4), no-guessing mandate (§11.4.6), credentials-handling mandate (§11.4.10), host-session safety (§12 + §12.6 + §12.10), and data safety (§9) all live in `constitution/Constitution.md`. Read it before working on any non-trivial change.

@constitution/QWEN.md

Canonical reference: https://github.com/HelixDevelopment/HelixConstitution

How this file relates to CLAUDE.md + AGENTS.md

Per §11.4.35 canonical-root inheritance clarity:

- `constitution/QWEN.md` is the universal canonical root for the Qwen Code CLI.
- This file is the consumer-side extension for this submodule, carrying only the inheritance pointer + the §11.4 covenant anchors + a brief module summary.

- The full module ruleset lives in this submodule's sibling `CLAUDE.md`. Qwen Code agents MUST read CLAUDE.md before performing any work; this QWEN.md is the Qwen-specific entry point that ensures Qwen reads the inheritance pointer + anti-bluff covenant on every session.

Module summary

Computer-vision module for HelixQA. Provides image/video analysis, OCR, frame-diff, and dual-display recording analysis used by the recording-analyzer + helixqa-bridge pipeline.

For full module context (build steps, integration points, host-session safety, submodule-specific commit/push discipline) read this directory's `CLAUDE.md` and `AGENTS.md`.

MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE (User mandate, 2026-04-28)

Forensic anchor — direct user mandate (verbatim):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This is the historical origin of the project's anti-bluff covenant. Every test, every Challenge, every gate, every mutation pair exists to make the failure mode (PASS on broken-for-end-user feature) mechanically impossible.

Operative rule: the bar for shipping is **not** "tests pass" but **"users can use the feature."** Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests AND Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end-user evidence; both are subject to the §8.1 five-constraint rule and §11 captured-evidence requirement.

Canonical authority: constitution submodule `Constitution.md` §11.4 (this end-user-quality-guarantee forensic anchor — propagation requirement enforced by pre-build gate `CM-COVENANT-114-QWEN-PROPAGATION`).

Non-compliance is a release blocker regardless of context.

Anti-bluff-manner clause (verbatim operator mandate — HRD-158 covenant cascade)

Forensic anchor — verbatim operator mandate:

> "IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well."

This clause is the leading verbatim operator anti-bluff mandate, restated here for cross-file consistency with this module's CLAUDE.md / AGENTS.md / CONSTITUTION.md. It does NOT weaken any inherited rule. Canonical authority: constitution submodule `Constitution.md` §11.4. Tests AND HelixQA Challenges are bound equally.

§11.4 extension anchors carried by this file

The following extension anchors apply unconditionally to every change landed in this submodule. Their full text lives in the sibling CLAUDE.md and in the canonical `constitution/Constitution.md`. Listed here so Qwen Code agents can locate them by literal string match:

- **§11.4.1 extension (Phase 33, 2026-05-05)** — FAIL-bluffs equally forbidden. A test that crashes for a script-internal reason and produces a FAIL exit code is just as misleading as a PASS-bluff. Fix at source layer, never at call sites.
- **§11.4.2 extension (Phase 34, 2026-05-06)** — Recorded-evidence requirement. Every PASS for a user-visible feature MUST be cross-checked by the analyzer against the dual-display recording + action timeline.
- **§11.4.3 extension (Phase 34, 2026-05-06)** — Per-device-topology test dispatch. Topology-touching tests MUST detect topology at entry and dispatch the topology-appropriate variant.
- **§11.4.4 extension (User mandate, 2026-05-06)** — Test-interrupt-on-discovery + retest-from-clean-baseline. The moment any defect is re-discovered or newly identified mid-cycle, STOP and fix at root cause + four-layer coverage + full rebuild + reflash + retest.
- **§11.4.4 expansion (User mandate, 2026-05-06)** — Systematic debugging via superpowers skills + four-layer test coverage per fix (pre-build gate + post-build gate + post-flash on-device test + HelixQA Challenge + paired mutation) + documentation update + no-bluff certification per cycle.
- **§11.4.5 — Audio + video quality analysis comprehensiveness (User mandate, 2026-05-07)** — Audio: presence + channel count + sample rate + glitch census + coexistence-artifact census. Video: presence + routing target + frame health + obstruction census + resolution + codec. Challenges bound equally.
- **§11.4.6 — No-guessing mandate (User mandate, 2026-05-08)** — Forbidden vocabulary: `likely`, `probably`, `maybe`, `might`, `possibly`, `presumably`, `seems`, `appears to`. Prove with captured evidence OR mark `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:`.

- **§11.4.7 — Demotion-evidence rule (Phase 38.X+2 amendment, 2026-05-11)** — Demotion from FAIL to lower-severity requires positive evidence captured under the same conditions that originally exposed the defect.
- **§11.4.8 — Deep-web-research-before-implementation mandate (User mandate, 2026-05-12)** — Cite external source URL OR literal "NO external solution found — original work" in every non-trivial fix.
- **§11.4.9 — Batch-source-fixes-before-rebuild mandate (User mandate, 2026-05-12)** — All source-side fixes that DO NOT require runtime on-device validation MUST be landed BEFORE the next firmware rebuild.
- **§11.4.10 — Credentials-handling mandate (User mandate, 2026-05-12)** — Credentials NEVER live in tracked files. `.env` git-ignored project-wide. Tests load from `scripts/testing/secrets/` (chmod 600).
- **§11.4.13 — Out-of-band sink-side captured-evidence mandate (User mandate, 2026-05-13)** — When an HDMI sink with network-accessible introspection API is present, every audio test MUST consume the sink's report as captured-evidence.
- **§11.4.14 — Test playback cleanup mandate (User mandate, 2026-05-13)** — Every test that issues `am start` / `cmd media_session play` MUST issue matching `am force-stop` / `KEYCODE_MEDIA_STOP` + EXIT trap.
- **§11.4.15 — Item-status tracking mandate (User mandate, 2026-05-13)** — Every active item in `Issues.md` carries a `**Status:**` line.
- **§11.4.16 — Item-type tracking mandate (User mandate, 2026-05-14)** — Every active item in `Issues.md` carries a `**Type:**` line with one of `{Bug | Feature | Task}`.
- **§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)** — Release tag MUST NOT be created until a complete retest with ALL existing tests has been executed on a clean baseline.
- **§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)** — Every force-push MUST be preceded by a mechanical 4-step merge-first pipeline (fetch + integrate + audit + push with `--force-with-lease`).

MANDATORY §12.6 MEMORY-BUDGET CEILING — 60% MAXIMUM (User mandate, 2026-04-30)

Project procedures MUST NOT use more than **60% of total system RAM** (`HOST_SAFETY_MAX_MEM_PCT`). The remaining 40% is reserved for the operator's other workloads. There is NO operator-facing override flag.

`scripts/build.sh` wraps `m -j` in `bounded_run` so only the bounded scope is OOM-killed if the build's collective RSS exceeds the budget — `user@<uid>.service` stays alive.

Canonical authority: constitution submodule `Constitution.md` §12.6.

Non-compliance is a release blocker regardless of context.

Companion documents

File	Role
`CLAUDE.md`	Full module ruleset (Claude Code primary context)
`AGENTS.md`	Cross-agent mirror (OpenCode, Cursor, Aider, generic AI tooling)
`QWEN.md` (this file)	Qwen Code CLI entry point
`../../../../constitution/Constitution.md`	Universal canonical rules
`../../../../constitution/QWEN.md`	Universal Qwen entry point

§11.4.69 — Universal Sink-Side Positive-Evidence Taxonomy + Mechanical Enforcement (cascaded from constitution submodule §11.4.69)

> Verbatim user mandate (2026-05-20): *"THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! ... Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! ... Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!"*

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (`audio_output`, `audio_input`, `video_display`, `network_throughput`, `network_connectivity`, `bluetooth_a2dp`, `bluetooth_pair`, `touch_input`, `sensor`, `gpu_render`, `storage_read`, `storage_write`, `mediacodec_decode`, `mediacodec_encode`, `miracast`, `cast`, `boot_service`, `package_install`, `permission_grant`, `wifi_link`, `wifi_throughput`, `ethernet_link`, `display_topology`, `drm_playback`, `subtitle_render` — open to additions, never contraction). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape. New helper contracts (additive during grace, mandatory after 2026-06-19): `ab_pass_with_evidence <description> <evidence_path>` (verifies path exists + non-empty), `ab_skip_with_reason <description> <closed-set-reason>` (reasons: `geo_restricted`, `operator_attended`, `hardware_not_present`, `topology_unsupported`, `network_unreachable_external`, `feature_disabled_by_config`; forbids `network_unreachable_external` for any taxonomy feature with a sink-side probe); bare `ab_pass` deprecated (WARN pre-grace, FAIL post-grace). Three pre-build gates + paired §1.1 mutations: `CM-SINK-EVIDENCE-PER-FEATURE`, `CM-NO-FAIL-OPEN-SKIP`, `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE`. No escape hatch — no `--skip-evidence`, `--config-only-pass`, `--allow-fail-open-skip`, `--legacy-ab-pass-permitted` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.69` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-69-PROPAGATION` enforces the anchor literal across the consumer fleet; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.69 for the full mandate.

§11.4.75 — Mechanical Enforcement Without Exception (cascaded from constitution submodule §11.4.75)

> Verbatim user mandate (2026-05-20): *"Why do these violations still happen!? This is a serious problem! We cannot rely on stability nor consistency if we cannot respect our Constitution, mandatory rules and constraints! Is there a way to make this always respected, followed and applied without exception fully and unconditionally!? WE MUST HAVE THIS WORKING FLAWLESSLY!!! Do investigate the root causes of such problems! Once all problems are identified WE MUST apply proper mechanisms for this not to happen NEVER EVER AGAIN!"*

The §11.4 covenant historically relied on agent + operator vigilance; three 2026-05-19→20 forensic incidents proved that late-binding enforcement fires hours-to-days after the violator commit reaches every remote. §11.4.75 closes the gap with FIVE independent mechanical enforcement layers — bypassing any single layer does not bypass the discipline: (1) local `pre-commit` git hook (refuses staged `.md` lacking sibling `.html`+`.pdf`); (2) `commit_all.sh` integration (`_constitution_sibling_check` + auto-`sync_all_markdown_exports.sh` self-repair); (3) local `pre-push` git hook (re-runs siblings + propagation-gate subset); (4) `post-commit` auto-repair hook (auto-generates orphan-`.md` siblings, idempotent + recursion-guarded); (5) local-only final-gate ritual (remote CI DISABLED per User mandate — operator runs `pre_build_verification.sh` + meta-test before every tag per §11.4.40). Helper contracts: `scripts/install_git_hooks.sh`, `scripts/git_hooks/{pre-commit,pre-push,post-commit,commit-msg}`, `_constitution_sibling_check`. The `commit-msg` hook enforces a `Bypass-rationale: <reason>` footer when `--no-verify` is detected; `docs/audit/bypass_events.md` accumulates the audit trail. Five gates with paired §1.1 mutations: `CM-COVENANT-114-75-PROPAGATION`, `CM-GIT-HOOKS-INSTALL-SCRIPT`, `CM-GIT-HOOKS-SOURCE-DIR`, `CM-COMMIT-ALL-SIBLING-CHECK`, `CM-CI-WORKFLOW-PRESENT`. No escape hatch — no `--skip-hooks`, `--bypass-enforcement`, `--allow-orphan-md`, `--ci-not-applicable`, `--mechanical-enforcement-not-needed` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.75` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-75-PROPAGATION`; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the enforcement layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.75 for the full mandate.

§11.4.76 — Containers-Submodule Mandate (cascaded from constitution submodule §11.4.76)

> Verbatim user mandate (2026-05-20): *"For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemy / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: https://github.com/vasic-digital/containers (git@github.com:vasic-digital/containers.git). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!"*

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), every consuming project MUST: (1) install `vasic-digital/containers` (`digital.vasic.containers`) as a Git submodule; (2) consume via `replace` directive during development + pinned commit SHAs in production; (3) boot infra on-demand via `pkg/boot` + `pkg/compose` + `pkg/health` so operators are never required to start `podman machine` / `docker compose up` manually — the boot is part of the test entry point (the on-demand-infra invariant); (4) extend the Submodule (PR upstream) for missing runtimes / lifecycle primitives — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule — short-circuit fakes that bypass boot are a §11.4 violation. Tracker rows touching containerization MUST record `Catalogue-Check: extend vasic-digital/containers@<sha>` (or `reuse`). Planned gate `CM-CONTAINERS-USED` scans container-touching PRs for `digital.vasic.containers/...` imports; paired mutation strips the import + asserts FAIL.

Cascade requirement: This anchor (verbatim or by `§11.4.76` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-76-PROPAGATION`; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule `Constitution.md` §11.4.76 for the full mandate.

§11.4.77 — Regeneration-Mechanism-Required Mandate (cascaded from constitution submodule §11.4.77)

> Verbatim user mandate (2026-05-20): *"We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content!"*

Every `.gitignore` entry excluding (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project MUST carry a documented + automated mechanism to either re-obtain (download from authoritative source: vendor tarball, SDK installer, npm/pip/cargo/go-mod/container registry, dedicated git submodule, S3/GCS) OR re-generate (run from tracked source via build pipeline, code-gen, asset render, captured-evidence replay, container build). Required artefacts per qualifying entry: (1) `.gitignore-meta/<entry-slug>.yaml` declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source +

integrity hash + requires-network + requires-credentials; (2) a non-interactive entry in `scripts/setup.sh` post-clone bootstrap; (3) a pre-build gate verifying regenerated content present OR a recent `.gitignore-meta/.regenerated/<slug>.ok` stamp; (4) README + `docs/guides/*.md` describing the mechanism + manual fallback + time/disk budget + §11.4.10 credentials. Bare `.gitignore` additions without the mechanism are a §11.4 PASS-bluff variant — codebase appears complete but a fresh clone cannot build/run. No escape hatch — no `--skip-regen-mechanism`, `--gitignore-is-enough`, `--operator-already-has-content` flag. Planned gate `CM-GITIGNORE-REGEN-MECHANISM` + paired §1.1 mutation (strip a required YAML key → gate FAILs).

Cascade requirement: This anchor (verbatim or by `§11.4.77` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-77-PROPAGATION`; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.77 for the full mandate.

§11.4.78 — CodeGraph Code-Intelligence Mandate (cascaded from constitution submodule §11.4.78)

> Verbatim user mandate (2026-05-20): *"Make codegraph MANDATORY CHOICE for this purpose for all of our project ... All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!"*

Every consuming project worked on by AI coding agents MUST install, initialize, and use **CodeGraph** (`https://github.com/colbymchenry/codegraph`, npm `@colbymchenry/codegraph`) — a local SQLite semantic code-knowledge-graph exposed to agents over MCP (100% local, no cloud). (1) Install globally via npm with a user-writable npm prefix (no `sudo`). (2) `codegraph init` + `codegraph index`: `.codegraph/config.json` is tracked, `.codegraph/codegraph.db` is gitignored with `codegraph index` as its §11.4.77 regeneration mechanism; the `config.json` `exclude` list MUST exclude every credential/secret path per §11.4.10. (3) Wire `codegraph serve --mcp` into every CLI agent (Claude Code `.mcp.json`, OpenCode `opencode.json`, Qwen Code `.qwen/settings.json`, Crush `.crush.json`, host-local otherwise) referencing the bare `codegraph` command on `PATH` (no hardcoded host path). (4) Cover the integration with an anti-bluff suite whose per-agent end-to-end layer uses an unforgeable challenge (a fact obtainable only by calling a CodeGraph MCP tool, e.g. index node count via `codegraph_status`); a genuinely un-drivable agent is a documented SKIP per §11.4.3, never a faked PASS. (5) Document in `docs/CODEGRAPH.md`, kept in sync per §11.4.12 / §11.4.65. CodeGraph is consumed as the published npm package (§11.4.74) — not a git submodule, adds no Git remote. Planned gate `CM-CODEGRAPH-WIRED` + paired §1.1 mutation (strip a secret-exclusion → gate FAILs).

Cascade requirement: This anchor (verbatim or by `§11.4.78` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-78-PROPAGATION`; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule `Constitution.md` §11.4.78 for the full mandate.

§11.4.79 — Own-Org Submodules MUST Be Included in the CodeGraph Index (cascaded from constitution submodule §11.4.79)

> Verbatim user mandate (2026-05-21): *"All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!"*

Refines §11.4.78's exclude-list with a per-submodule-ownership split: (a) own-org submodules (full write access via the project's CLIs — canonical orgs `vasic-digital` + `HelixDevelopment`) MUST be INCLUDED in the index; (b) third-party submodules (the §11.4.74 `no-match → vendor` path) MUST be EXCLUDED. Operational steps: (1) `git submodule update --remote --merge` to pull latest before re-indexing, respecting load-bearing pins on third-party submodules; (2) adjust `.codegraph/config.json` exclude list to keep own-org paths in scope; (3) re-index via `scripts/codegraph_setup.sh`; (4) verify via `scripts/codegraph_validate.sh` with ≥1 probe resolving a symbol living ONLY inside an own-org submodule; (5) paired §1.1 mutation — temporarily add the own-org submodule to exclude → validate MUST FAIL on the cross-submodule probe → restore. An index that lies about reachable symbols is a PASS-bluff against AI agents. Own-org submodules silently excluded without an audit trail in `.codegraph/config.json` comments is a release blocker.

Cascade requirement: This anchor (verbatim or by `§11.4.79` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-79-PROPAGATION`; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule `Constitution.md` §11.4.79 for the full mandate.

§11.4.80 — CodeGraph Regular-Update + Sync Automation Mandate (cascaded from constitution submodule §11.4.80)

> Verbatim user mandate (2026-05-21): *"We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed on the host machine! ... Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule ... Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents ... and regularly export them like all other Status docs into the PDF and HTML!"*

Three deliverables (all living in the constitution submodule, inherited by reference per §3 — consuming projects invoke at `${CONST_DIR}/scripts/codegraph_*.sh`, never copy): (1) `scripts/codegraph_update.sh` — npm-installs latest `@colbymchenry/codegraph` after a registry version check; appends old/new version to `docs/codegraph/Status.md`; anti-bluff verifies `codegraph --version` reflects the new version after install (npm exit 0 ≠ working binary). (2) `scripts/codegraph_sync.sh` — after a successful update runs `codegraph status` → `codegraph sync .` → `codegraph status` → the project's `scripts/`

codegraph_validate.sh; appends every step's output to BOTH the project's and the constitution's docs/codegraph/Status.md. (3) docs/codegraph/Status.md + Status_Summary.md append-only ledgers, exported to .html + .pdf per §11.4.65. Cadence: weekly floor (per §11.4.45). A consuming project that has not run codegraph_update.sh in >2 weeks AND has open AI-agent work is a release blocker. Paired §1.1 mutation: downgrade installed version → script detects drift → restore.

Cascade requirement: This anchor (verbatim or by §11.4.80 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-80-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule Constitution.md §11.4.80 for the full mandate.

§11.4.81 — Cross-Platform-Parity Mandate (cascaded from constitution submodule §11.4.81)

> Verbatim user mandate (2026-05-21): *"Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!"*

Every consuming project whose supported-platforms manifest lists more than one OS MUST, for every feature/test/gate/challenge/mutation depending on platform-specific primitives, ship a per-OS-equivalent implementation chosen at runtime via uname -s (or equivalent detection). Three sub-mandates: **(A) Per-OS implementation REQUIRED** — Linux cgroup/systemd/proc primitives MUST have documented per-OS equivalents (POSIX setrlimit/ulimit, macOS launchd, BSD rctl, Windows Job Object) chosen via runtime dispatch. **(B) Per-OS tests REQUIRED** — every platform-dependent gate test MUST have case "$(uname -s)" in branches with positive captured evidence per §11.4.2 + §11.4.5 in each branch; SKIP-with-reason acceptable ONLY when the platform genuinely cannot enforce the invariant. **(C) Honest kernel-gap citation + adjacent equivalent test REQUIRED** — where a Linux primitive has NO equivalent due to a documented kernel limitation (canonical: XNU does not enforce RLIMIT_AS for unprivileged processes), the test MUST detect the gap at runtime, SKIP with exact kernel reason + reproducer + honest-gap-doc link, AND provide an ADJACENT test exercising the closest invariant the platform CAN enforce (e.g. RLIMIT_CPU+SIGXCPU as the macOS proxy), itself anti-bluff with a paired §1.1 mutation. Gate CM-CROSS-PLATFORM-PARITY scans for case "$(uname -s)" blocks asserting a non-SKIP branch (or honest-gap citation) per platform in the manifest; paired mutation strips a Darwin branch → gate FAILs. No escape hatch.

Cascade requirement: This anchor (verbatim or by §11.4.81 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-81-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker on multi-platform projects. **Canonical authority:** constitution submodule Constitution.md §11.4.81 for the full mandate.

§11.4.82 — Iteration-Speedup Discipline Mandate (cascaded from constitution submodule §11.4.82)

> Verbatim user mandate (2026-05-22): *"How can we speed-up this whole development and fixing process? … Do not forget to all speed optimizations critical rules and mandatory constraints MUST BE all added into our root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md and QWEN.md and all other relevant constitution Submodules files!"*

Iteration cycle time is a first-order quality enabler. Every consuming project's build / test / commit / debug pipeline MUST adopt these speedup disciplines AS MANDATORY (each independently enforceable): (A) Phase-1 forensic (`superpowers:systematic-debugging`) before any speculative source patch — speculative patches without FACT-grade root cause are §11.4.6 + §11.4.82 violations; (B) Live-ADB-First (or live-equivalent) before any rebuild — strengthens §11.4.51 to a release-blocker mandate; (C) 30-second pre-flight before launching rebuild orchestrators (device/sink reachability, host memory/disk, no stale locks, no orphan processes); (D) persistent build caches outside containers (`ccache`/`sccache`/Gradle daemon bind-mounted to host); (E) module-only rebuild for loadable-module-only changes; (F) parallel multi-device testing with separate `qa-results/<TS>/<device-tag>/` outputs; (G) subagent scope discipline + worktree isolation (≤30 min budget, single-responsibility, `isolation: "worktree"` default); (H) lock-file + stale-process hygiene (clean `.git/index.lock`, disable auto git-gc in concurrent repos); (I) cycle telemetry per §11.4.24 (commit hash, per-phase wall-clock, speedup-flag set, outcome — aggregated weekly). Gate `CM-ITERATION-SPEEDUP-DISCIPLINE` audits recent cycles for telemetry citing which of (A)-(I) applied; paired §1.1 mutation strips the speedup-flag column → gate FAILs. No escape hatch — no `--skip-phase1-forensic`, `--no-pre-flight`, `--rebuild-everything-always`, `--unlimited-subagent-scope`, `--ignore-locks`, `--no-telemetry` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.82` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-82-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.82 for the full mandate.

§11.4.83 — docs/qa/ End-User Evidence Mandate (cascaded from constitution submodule §11.4.83)

> Verbatim user mandate (2026-05-22): *"every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under `docs/qa/<run-id>/` (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof."*

Every feature that ships MUST carry a recorded end-to-end communication transcript plus any attached materials (screenshots, request/response payloads, audio, file uploads) committed under `docs/qa/<run-id>/` — one directory per feature run. Operative rule: (1) every consuming project MUST maintain a `docs/qa/` tree, each new feature under `docs/qa/<run-id>/` where `<run-id>` is

monotonic + greppable (timestamp / ATM-NNN / other workable-item ID per §11.4.54); (2) transcripts MUST be full bidirectional — every prompt/command sent + every response received (one-sided is not a transcript); (3) attached materials MUST be committed in-repo (no external-only links — that is a §11.4.13 sink-side violation); (4) bot-driven / agent-driven QA automation MUST preserve the full conversation thread as the proof artefact; (5) release gates MUST refuse to tag a version that has any feature-shipping commit without its matching `docs/qa/<run-id>/` directory. A feature with no QA transcript is a §11.4 / §107 PASS-bluff. Composes with §11.4.2 / §11.4.5 / §11.4.13 / §11.4.65 / §11.4.69 / §1.1.

Cascade requirement: This anchor (verbatim or by `§11.4.83` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-83-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker — no `--qa-evidence-optional` escape hatch. **Canonical authority:** constitution submodule `Constitution.md` §11.4.83 for the full mandate.

§11.4.84 — Working-Tree Quiescence Rule for Subagent Commits (cascaded from constitution submodule §11.4.84)

> Verbatim user mandate (2026-05-22): *"no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before `git add`, the committing agent MUST `grep` its own working tree for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT."*

No subagent (or main-thread) commit may proceed while any concurrent mutation gate, paired-mutation experiment, or other in-flight mutation is live in the same checkout. Before `git add`, the committing agent MUST grep its own working tree for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcut paths, `// MUTATION` / `# MUTATION` annotations, `_mutated_*` filename suffixes, etc.) and explicitly account for every modified file in the staging area; any unexplained file → ABORT. (Forensic case: a logo-fix subagent's `git add` swept an `// always pass` JWT-verify mutation residue into an unrelated commit pushed to all four mirrors — a real security-defect window.) Operative rule: (1) pre-`git add` greps for mutation markers + cross-checks `git status --porcelain` against the subagent's declared scope; unaccounted entries → ABORT; (2) any active mutation gate MUST be serialised (mutate → assert FAIL → restore → assert PASS) and the working tree verifiably clean before any unrelated commit; (3) concurrent subagents in the SAME checkout MUST coordinate through a lockfile (`.git/MUTATION_IN_PROGRESS`) — cleaner solution is `git worktree add` per subagent (composes with §11.4.20/§11.4.70); (4) post-commit `mutation-residue-scanner` MUST run before push — any commit containing a mutation marker → push BLOCKED.

Cascade requirement: This anchor (verbatim or by `§11.4.84` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-84-PROPAGATION`; paired mutation strips the literal → gate FAILs. A mutation marker that lands in a tagged

commit is a critical defect regardless of how briefly it persisted. **Canonical authority:** constitution submodule `Constitution.md` §11.4.84 for the full mandate.

§11.4.85 — Stress + Chaos Test Mandate (cascaded from constitution submodule §11.4.85)

> Verbatim user mandate (2026-05-24): *"Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"*

Every fix or improvement landed MUST ship with full-automation **stress** AND **chaos** test suites exercising edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 PASS-bluff at the resilience layer. **Stress** (closed-set): sustained load (N ≥ 100 iterations OR ≥ 30 s wall-clock, p50/p95/p99 latency recorded) + concurrent contention (N ≥ 10 parallel invocations, no deadlock/leak) + boundary conditions (empty/max/off-by-one, each categorised). **Chaos** (closed-set, per fix-class appropriateness): process-death injection + network-fault injection (drop/delay/reorder) + input-corruption injection + resource-exhaustion injection (disk full, OOM, FD exhaustion — refuse cleanly OR degrade, NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write). Every stress + chaos PASS MUST cite a captured-evidence artefact path per §11.4.5 + §11.4.69. Helper library `stress_chaos.sh` provides `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`, `ab_chaos_drop_network_during`, `ab_chaos_corrupt_file_during`, `ab_chaos_oom_pressure_during`, `ab_chaos_disk_full_during`, each composing with `ab_pass_with_evidence` / `ab_skip_with_reason`. Cleanup non-negotiable in `trap '...' EXIT` (cleanup failure = §11.4.14 violation). Four-layer coverage per §11.4.4(b) + paired §1.1 mutation (strip chaos-injection or evidence-capture → gate FAILs). No escape hatch — no `--skip-stress`, `--no-chaos`, `--happy-path-suffices`, `--stress-test-later` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.85` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-85-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.85 for the full mandate.

§11.4.86 — Roster/Corpus-Backed Status-Doc Auto-Sync Mandate (cascaded from constitution submodule §11.4.86)

> Verbatim user mandate (2026-05-25): *"Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!"*

Some Status docs (§11.4.45) are backed by a tracked roster (installed apps/components) or a tracked asset corpus (test/media asset directory) rather than narrative alone. Their freshness MUST NOT depend on operator vigilance — the moment a roster/corpus member changes (app added/removed/renamed; asset added/modified/removed) the Status doc + Status_Summary + HTML + PDF MUST resync out of the box, mechanically. Mechanism (all must hold): (1) drift-proof fingerprint — sha256 of the sorted member list (NOT mtime), persisted in a sidecar beside the Status doc; (2) a sync helper that regenerates the fingerprint + re-exports HTML+PDF via the §11.4.65 exporter, wired so sync is automatic; (3) a pre-build gate that FAILs when the live fingerprint differs from the persisted one (mirrors §11.4.12 `CM-ISSUES-SUMMARY-SYNC` + §11.4.45 `sync_integration_status`); (4) a paired §1.1 mutation corrupting the fingerprint and asserting the gate FAILs. Classification: universal — the consuming project supplies the specific docs, roster/corpus sources, helper, and gate name per §11.4.35.

Cascade requirement: This anchor (verbatim or by `§11.4.86` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-86-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker — no `--skip-roster-sync`, `--allow-status-drift`, `--roster-sync-not-applicable` flag. **Canonical authority:** constitution submodule `Constitution.md` §11.4.86 for the full mandate.

§11.4.87 — Endless-Loop Autonomous Work + Zero-Idle Agent Dispatch + Anti-Bluff Testing Mandate (cascaded from constitution submodule §11.4.87)

> Verbatim user mandate (2026-05-26): *"continue in endless loop fully autonomously"* (and any semantically-equivalent phrasing).

When the operator instructs an AI agent to continue in an endless autonomous loop, the agent MUST treat it as a HARD-CONTRACT covenant: (A) continue working until `docs/Issues.md` Status-column has zero non-terminal entries AND `docs/CONTINUATION.md` §3 Active work is empty AND no background subagent is mid-execution AND no external dependency is in-flight; (B) dispatch background subagents for parallelisable work — main + every subagent operate concurrently, "waiting for results" is the ONLY acceptable idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence (audio/video/network/UI/sysfs physical proofs); (D) the §11.4 anti-bluff covenant family (§11.4.1 / §11.4.2 / §11.4.6 / §11.4.7 / §11.4.27 / §11.4.50 / §11.4.52 / §11.4.68 / §11.4.69 / §11.4.83) is the operative truth-discipline — tests AND HelixQA Challenges bound equally; (E) the loop terminates ONLY on all-conditions-met, explicit operator STOP, host-session-safety demand, or scheduled wake on a known-future-actionable signal. No escape hatch — no `--idle-OK`, `--skip-endless-loop`, `--bluff-permitted-for-this-task`, `--metadata-only-test-suffices`, `--no-physical-proof-required` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.87` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-87-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.87 for the full mandate.

§11.4.88 — Background-Push Mandate: Commit-Lock Release Immediately After Commit, Push Runs Detached (cascaded from constitution submodule §11.4.88)

Forensic anchor (2026-05-26): a single `commit_all.sh` held its flock ~5 hours because `do_push` ran synchronously after the commit landed — every subsequent commit blocked on a slow mirror push irrelevant to the local commit's durability. Implementation seam for §11.4.87(B) zero-idle. The mandate: (A) `.git/.commit_all.lock` MUST be released IMMEDIATELY after `git commit` returns 0 — the commit is durable on local disk regardless of remote push outcome; (B) push runs detached via `nohup ./push_all.sh ... > <log> 2>&1 &` + `disown` — the orchestrator's exit code reports COMMIT success, NOT push success; (C) `push_all.sh` acquires per-remote flock `.git/.push.<remote>.lock` so concurrent invocations targeting the same remote serialize but different-remote invocations run in parallel; (D) backgrounded push failures land in `qa-results/push_failures/<ts>_<remote>.log` — the next autonomous-loop tick checks per §11.4.87(A) "no external dependency in-flight" gate; (E) synchronous-push escape: explicit `--sync-push` CLI flag preserves legacy behaviour for §11.4.41 force-push merge-first audit paths. Gates `CM-COVENANT-114-88-PROPAGATION` + `CM-BACKGROUND-PUSH-WIRED` + paired §1.1 mutations. Synchronous push (without `--sync-push`) = §11.4 PASS-bluff at the execution layer.

Cascade requirement: This anchor (verbatim or by `§11.4.88` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-88-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker — no escape hatch beyond `--sync-push` for force-push events. **Canonical authority:** constitution submodule `Constitution.md` §11.4.88 for the full mandate.

§11.4.89 — Background Test Execution Mandate (cascaded from constitution submodule §11.4.89)

> Verbatim user mandate (2026-05-27): *"Any tests we are executing, especially long test cycles, MUST BE performed in background in parallel with main work stream! This MUST NOT block our capabilities to work on queued workable items. Main work stream can be blocked or sit iddle only if absolutely needed and if it depends hard on results of some background execution."*

Symmetric anchor to §11.4.88 (background push) at the test-execution layer. Mandate: (A) long-running tests (>30 s expected: `pre_build`, `meta_test`, `test_all_fixes`, `recent_work_validate`, HelixQA banks, 4-phase cycles, full-suite retests, audio supervisors, dual-display recorders) MUST run via `nohup ... > <log> 2>&1 &` + `disown` with the log under a known dir (`qa-results/<test_id>_<ts>.log`); (B) the main stream proceeds to the §11.4.42 priority queue immediately; (C) hard-dependency gating — poll an exit-status file or `pgrep -af <test>` before steps that need the exit code, surfacing as §11.4.66 interactive options if the test is still running; (D) failures land in `<log>` files, the next loop tick checks; (E) foreground execution permitted ONLY for <30 s tests OR

explicit operator authorisation; (F) per-script flock serialises same-script invocations, different-script invocations parallel. Gates `CM-COVENANT-114-89-PROPAGATION` + `CM-BACKGROUND-TEST-EXECUTION-WIRED` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.89` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-89-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker — no escape hatch beyond explicit per-invocation operator authorisation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.89 for the full mandate.

§11.4.90 — Obsolete Status + Per-Item Obsolescence Audit (cascaded from constitution submodule §11.4.90)

> Verbatim user mandate (2026-05-27): *"Bug No 6 ... seems obsolete after latest request for new behavior ... mark obsolete tickets with some light gray background ... text - the description to be strikethrough styled ... review all existing open or resolved workable items if they are obsolete - not valid any more ... There MUST NOT be any mistake! No bluff is allowed of any kind!"*

The §11.4.15 Status closed-set is extended with a terminal `Obsolete (→ Fixed.md)` value (orthogonal to Type per §11.4.16). Obsolescence reasons (closed vocabulary): `superseded-by-design-change | superseded-by-later-mandate | feature-removed | duplicate-of | unsupported-topology`. Every Obsolete heading MUST carry an `**Obsolete-Details:**` line (Since + Reason + Superseding-item + Triple-check evidence) within 8 non-blank lines. The §11.4.23 colorizer adds a `cell-status-obsolete` class — light-gray `#E0E0E0` background + strikethrough description. Audit cadence: every release-gate sweep per §11.4.40 + §11.4.42; triple-check is non-negotiable per the operator mandate. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.21 / §11.4.23 / §11.4.33 / §11.4.34 / §11.4.40 / §11.4.42 / §11.4.66 / §11.4.71. Gates `CM-COVENANT-114-90-PROPAGATION` + `CM-ITEM-OBSOLETE-DETAILS` + `CM-OBSOLETE-COLORIZER-WIRED` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.90` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-90-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.90 for the full mandate.

§11.4.91 — Summary-Doc Clarity Mandate (cascaded from constitution submodule §11.4.91)

> Verbatim user mandate (2026-05-27): *"Summary docs - Issues_Summary some not clear one line descriptions - like 'Composes with' ... For each workable item we MUST HAVE clearly understandable meaning ... every*

> *team member can clearly understand what that particular workable item is exactly about! There cannot be misunderstanding or unclearity of any kind and no bluff allowed!"*

Every summary entry (Issues_Summary, Fixed_Summary, README doc-link, Status_Summary pages 1+2, all one-liners) MUST contain a self-contained meaningful description ≥ 6 words OR ≥ 40 chars naming SUBJECT + PROBLEM/GOAL. Forbidden one-liner anti-patterns: section labels (`Composes with`, `Closure criteria`, `Fix direction`, etc.); bare metadata fragments (`Critical`, `Bug`, `In progress`, etc.); section-marker echoes; a §-letter alone. Generators (`generate_issues_summary.sh` / `generate_fixed_summary.sh` / `update_readme_doc_links.sh` / `generate_status_summary.sh`) MUST extract from the H1/H2 heading line per the §11.4.54 ATM-NNN convention, NEVER from arbitrary downstream text, and MUST refuse anti-pattern rows — emitting a `(MISSING DESCRIPTION — fix source heading)` placeholder with visual highlight. Gate `CM-SUMMARY-CLARITY-DESCRIPTIONS` scans every summary; an anti-pattern match = FAIL. Audit cadence: every §11.4.40 + §11.4.42 sweep.

Cascade requirement: This anchor (verbatim or by `§11.4.91` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-91-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.91 for the full mandate.

§11.4.92 — Multi-Pass Change-Evaluation Discipline (cascaded from constitution submodule §11.4.92)

> Verbatim user mandate (2026-05-27): *"Every change to the project or codebase we do MUST BE evaluated in several passes and in in-depth analisys for potential new issues or problems it can introduce! ... no bluff of any kind! After we do change or set of changes this mandatory steps MUST BE taken!"*

Every non-trivial change MUST pass a 5-pass evaluation BEFORE it is commit-ready: **(Pass 1)** main-task verification — change achieves the stated goal, captured-evidence per §11.4.5/§11.4.69; **(Pass 2)** regression-blast-radius analysis — enumerate every direct dependency, demonstrate no contract break; **(Pass 3)** cross-feature interaction analysis — audit parallel features sharing state/timing/hardware/shell environment; **(Pass 4)** deep-research validation per §11.4.8 — external precedent OR "NO external solution found — original work" + CodeGraph queries per §11.4.78/§11.4.79; **(Pass 5)** anti-bluff confirmation per §11.4 / §11.4.1 / §11.4.6 / §11.4.27 / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.83 — no new bluff surface introduced. Each pass is documented (commit footers OR `docs/` entries OR `qa-results/` evidence). Only after all 5 passes complete may commit/push/test/release proceed. Trivial exemption: typo / revision-bump / MD-export-regen IF zero source touched AND the commit message cites the exemption explicitly. Gates `CM-COVENANT-114-92-PROPAGATION` + `CM-MULTI-PASS-EVALUATION-EVIDENCE` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.92` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-92-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.92 for the full mandate.

§11.4.93 — SQLite-Backed Single-Source-of-Truth for Workable Items (cascaded from constitution submodule §11.4.93)

> Verbatim user mandate (2026-05-27): *"There MUST be single source of truth for all of our workable items - SQlite database ... proper scripts (we recommend Go programs) ... reduce a chance for sync to be broken ... generate always all docs from DB or to re-generate Db from all docs we have in opposite direction"*

The text-based Issues/Fixed/Summary/CONTINUATION constellation is converted to a SQLite-DB-backed single source of truth. Schema mandatory tables: `items` (atm_id PK + Type + Status incl. Obsolete + Severity + title + description ≥40 chars + created/modified + composes_with JSON + current_location); `item_history` (append-only audit per §11.4.34 By/Reason/Evidence); `obsolete_details` (§11.4.90); `operator_block_details` (§11.4.21); `firebase_metadata` (§11.4.47); `meta` (schema version + last sync + integrity hash). A Go binary at `cmd/workable-items/` provides `sync` `md-to-db` / `db-to-md` / `diff` / `validate` / `add` / `close`; bidirectional regen is byte-identical round-trip (closed-set whitespace/section-order tolerance). `commit_all.sh` refuses on non-empty diff; `sync_issues_docs.sh` invokes the Go binary; pre-build runs `workable-items validate`. Anti-bluff: unit + integration + stress (1000-row insert + 10 concurrent writers) + chaos (mid-write SIGKILL + corrupt-DB recovery + disk-full) + paired §1.1 mutation + HelixQA Challenge `CME-WORKABLE-ITEMS-001`. The Go binary lives in the constitution submodule (`constitution/scripts/workable-items/`) per §11.4.74. Gates `CM-COVENANT-114-93-PROPAGATION` + `CM-WORKABLE-ITEMS-DB-PRESENT` + `CM-WORKABLE-ITEMS-MD-DB-IN-SYNC` + paired §1.1 mutations. (NOTE: the DB tracking rule is AMENDED by §11.4.95 — DB is TRACKED, not gitignored.)

Cascade requirement: This anchor (verbatim or by `§11.4.93` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-93-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker — text-based-only trackers are a §11.4 PASS-bluff at the data-architecture layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.93 for the full mandate.

§11.4.94 — Zero-Idle Priority-First Parallel-By-Default Operating Mode (cascaded from constitution submodule §11.4.94)

> Verbatim user mandate (2026-05-27): *"We MUST NEVER sit iddle / wait or sleep if there is possibility for us to work on something ... Always check if there is a possibility to work on something while we are not working actively*

> *on something! Pick always by priority - most critical workable items and other tasks MUST BE done first! … Stay still / iddle if nothing is left to be done at all or waiting for something that is blocking us / you!!!"*

§11.4.94 binds §11.4.20 + §11.4.42 + §11.4.58 + §11.4.70 + §11.4.72 + §11.4.82 + §11.4.87 + §11.4.88 + §11.4.89 into a single always-on enforcement: (A) idle ONLY when every queued item is genuinely blocked on an external dependency (hardware / network upstream / build/test completion the conductor cannot accelerate) OR operator STOP OR §12 host-safety — "don't see what to do" is NEVER valid; (B) before ANY wake/sleep the conductor MUST survey parallel-work feasibility per §11.4.42 + §11.4.72 + §11.4.87, identify non-contending items, and dispatch in parallel per §11.4.20/§11.4.70 (subagent) + §11.4.58 (PWU disjoint scope) + §11.4.89 (background long tests); (C) priority order MANDATORY — pick highest-severity + §11.4.72 audio-first the conductor can autonomously progress; (D) subagent-driven default for non-trivial; (E) background default for >30 s wall-clock work via `nohup`+`disown`; (F) stability-preserving (composes with §11.4.92 multi-pass + §11.4.84 quiescence + §12.6–§12.9 host safety); (G) progress updates surfaced at milestone boundaries. Gates `CM-COVENANT-114-94-PROPAGATION` + `CM-PARALLEL-WORK-AUDIT` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.94` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-94-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.94 for the full mandate.

§11.4.96 — Safe-Parallel-Work-With-Long-Build Catalogue + Mandate (cascaded from constitution submodule §11.4.96)

> Verbatim user mandate (2026-05-27): *"Are there except AOSP build process any other active jobs being done at the moment? Can we work on something in parallel while build is in progress so we slowly cleanup our slate? … do as much as possible work in background in parallel with main work stream and oreferrably using subagents-driven approach!"*

An operational catalogue for the canonical long-running workload (multi-hour containerised build per §12.9). **SAFE during build:** (A) MD/docs work; (B) generator/helper script work under `scripts/`; (C) pre-build + meta-test gate authoring + paired §1.1 mutations; (D) on-device test scripts; (E) constitution submodule edits + push; (F) any submodule commit + push per §11.4.88; (G) read-only live-ADB probes (`dumpsys`/`getprop`/`cat /proc/...`/`screencap`/`logcat`); (H) subagent dispatch per §11.4.20/§11.4.70 + §11.4.84 quiescence; (I) web research + external API queries with §11.4.10 credentials; (J) workable-items DB ops per §11.4.93+§11.4.95; (K) backgrounded pre-build + meta-test execution per §11.4.89. **UNSAFE during build:** (α) `git checkout`/`reset --hard`/`clean -df` on the source tree (use `git worktree`); (β) mass file deletes/renames under built source trees; (γ) submodule pointer updates affecting built artefacts; (δ) `out/` mutations; (ε) `make clean`/`m clobber`/`rm -rf out/`; (ζ) container destruction; (η) disk-filling breaching §12.9 free-space minimum; (θ) §12 host-session-safety breaches. Conductor responsibility: before EVERY pause point during a long build, consult the catalogue, identify (A)-(K) queue items per §11.4.42+§11.4.72, and dispatch ≥1

per §11.4.20/§11.4.70 subagent default + §11.4.89 background. “Build running, nothing else to do” is NEVER true per §11.4.94+§11.4.96. Gates `CM-COVENANT-114-96-PROPAGATION` + `CM-PARALLEL-WORK-DURING-BUILD-AUDIT` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.96` reference) MUST appear in every owned submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-96-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.96 for the full mandate.

§11.4.97 — Maximum-Use-of-Idle-Time + Progress-Update Cadence (cascaded from constitution submodule §11.4.97)

> Verbatim user mandate (2026-05-27): *“keep it working, we should do as much as possible, if not it all but as much as we can as long as there is iddle time! it MUST be used! … keep us updated about all progress and all phisycal proofs and gathered data as you progress through all open workable items!”*

Operating-mode capstone strengthening §11.4.87 + §11.4.94 + §11.4.96: (A) every minute of conductor idle time during which work could autonomously progress AND is not genuinely blocked = a §11.4.97 violation; “as much as possible, if not it all but as much as we can” is operative — dispatch CONTINUOUSLY through the entire idle window, not just at scheduled wakes; (B) progress-update cadence — emit an operator-facing 1-line update at every commit landed / subagent return / constitutional anchor / captured evidence / milestone closure, no operator prompt required; (C) continuous physical-proof gathering per §11.4.5 + §11.4.6 + §11.4.69 — every autonomous closure cites captured-evidence (evidence path goes into the §11.4.93 `item_history.evidence_path` when the DB lands); (D) composes with §11.4.5/6/13/20/27/42/50/52/69/70/72/83/85/87/88/89/94/96; (E) the idle-only-when-blocked closed-set is unchanged from §11.4.94(A). Gates `CM-COVENANT-114-97-PROPAGATION` + `CM-IDLE-TIME-AUDIT` + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by `§11.4.97` reference) MUST appear in every owned submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-97-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.97 for the full mandate.

§11.4.95 — Workable-Items SQLite DB Is TRACKED in Git, NEVER Gitignored (cascaded from constitution submodule §11.4.95)

> Verbatim user mandate (2026-05-27): *“We shall not Git ignore our workable items SQlite DB since it is our single source of truth … workable items SQlite DB regularly commited and pushed to all upstreams!”*

§11.4.93's earlier "gitignored per §11.4.30" clause is AMENDED — the DB at `docs/workable_items.db` is TRACKED in git, NEVER gitignored. It IS authoritative source data, NOT a build artefact. Every `workable-items sync md-to-db` that mutates state MUST stage + commit + push the DB alongside the MD regen per §11.4.19 atomic-move + §2.1 multi-upstream push. A WAL-checkpoint (`PRAGMA wal_checkpoint(TRUNCATE)`) is required before commit-stage so the transient `.db-wal` + `.db-shm` sidecars (gitignored per §11.4.30) are safely discardable. The §11.4.77 regeneration mechanism does NOT apply — the DB IS the source. Destructive DB ops require §9.2 hardlinked-backup + operator authorization; §11.4.41 force-push merge-first applies if DB history ever needs rewrite. Gates `CM-COVENANT-114-95-PROPAGATION` + `CM-WORKABLE-ITEMS-DB-TRACKED` + paired §1.1 mutation.

Cascade requirement: This anchor (verbatim or by `§11.4.95` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-95-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.95 for the full mandate.

---

§11.4.98 — Full-Automation Anti-Bluff Mandate (cascaded from constitution submodule §11.4.98)

> Verbatim user mandate (2026-05-28): *"Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun endless times when needed! ... Make sure there is no false positives in testing! Every test and its results MUST obtain real proofs of everything working! No bluff is allowed!"*

Closes the manual-intervention gap (§11.4 / §11.4.2 / §11.4.5 / §11.4.50 / §11.4.85 / §11.4.87 / §11.4.89 / §11.4.94 did not explicitly forbid it). A live/integration/e2e/Challenge test that requires a human action during execution (typing a message, clicking UI, hand-triggering a webhook, attaching a file — anything beyond startup) is by definition a §11.4 PASS-bluff at the automation layer. (A) Every governed test — unit/integration/e2e/Challenge/stress/chaos/live — MUST be fully self-driving end-to-end, reporting PASS/FAIL/SKIP-with-reason without any further human action after startup. (B) Single permissible exception: one-time credential bootstrap performed OUTSIDE test execution (`.env` from vault, shell exports, OAuth at first install, MTProto session activation) — configuration, not test driving. (C) Live messenger/channel/agent tests: no "operator must type" prompts (drive programmatically via second account / webhook fixture / loopback); no hard-coded session UUIDs that collide with the active dev session (Herald 2026-05-28 `claude --resume` silent exit -1 lesson); no 60 s human-response windows (§11.4.50 determinism violation); re-runnability proof — PASS at `-count=3` consecutive automated invocations with self-cleaning state; §11.4.98 obsolescence audit classifies every existing test COMPLIANT vs NON-COMPLIANT; no silent-skip-reported-as-PASS or stale-evidence-as-fresh. (D) With §11.4.85 + §11.4.89 + §11.4.87 + §11.4.94 forms a continuously-validated, non-flake, anti-bluff regime. (F) Manual-dependency tests not rewritten within 30 days graduate to §11.4.90 Obsolete citing §11.4.98.

Cascade requirement: This anchor (verbatim or by `§11.4.98` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-98-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.98 for the full mandate.

---

§11.4.99 — Latest-Source Documentation Cross-Reference Mandate (cascaded from constitution submodule §11.4.99)

> Verbatim user mandate (2026-05-28): *"Make sure we ALWAYS check against latest versions of services we use web / online docs before creating instructions! This situation is illustration of how we can misguide ourselves or get banned! ... These are mandatory rules / constraints and the result is consistency and safety of created instructions, guides and manuals!"*

Misguidance-by-stale-docs is the same severity class as a §11.4 PASS-bluff at the documentation layer (Herald 2026-05-28 case: a first-draft MTProto guide recommended VoIP fallback numbers and omitted the `recover@telegram.org` pre-login email — both contradicted Telegram's official docs + the gotd/td maintainer guide and could have caused a permanent account ban). Closes the gap §11.4.92 Pass 4 alludes to but does not mandate. (A) Before committing any operator-facing instruction/guide/manual/troubleshooting/setup doc, the author MUST: (1) fetch the LATEST official online documentation of the documented service/library via WebFetch / MCP / direct browsing — NEVER training data, memory, or prior committed docs; (2) cross-reference every instruction step against that source; (3) seek secondary authoritative sources (maintainer SUPPORT.md, official changelogs, vetted community FAQs) when the official source is sparse/silent; (4) cite source URLs + date in a `## Sources verified` footer in the doc; (5) cite a `Sources verified <date>: <urls>` footer in the commit message. (B) Negative findings (gaps/silences/contradictions) MUST be documented explicitly. (C) Docs older than 6 months are STALE — re-verify before citing as operator authority, at every vN.0.0 release boundary, on service breaking-change announcements, or on operator error reports. (D) Risk-classified services (messengers, cloud APIs, payment systems, AI/LLM providers, code-hosting, package managers) carry a 90-day max staleness + explicit safety warnings. (E) Composes with but is INDEPENDENT of §11.4.92 Pass 4. (G) Commit missing either footer is BLOCKED at release-gate; stale-beyond-grace docs graduate to §11.4.90 Obsolete (`Reason=stale-documentation`).

Cascade requirement: This anchor (verbatim or by `§11.4.99` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-99-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.99 for the full mandate.

---

§11.4.101 — Autonomous-Decision-Over-Blocking Mandate (cascaded from constitution submodule §11.4.101)

> Verbatim user mandate (2026-05-28): *"when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven't answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!"*

In autonomous / endless-loop mode (per §11.4.87), the agent MUST minimize operator-blocking and make the safe, reliable, reversible decision itself so work is not stalled (e.g. overnight) waiting for input — §11.4.87 says keep working, §11.4.101 says HOW to clear the decision points. **Proceed-autonomously (closed-set, ALL must hold):** (a) the action is reversible OR has a captured pre-op backup per §9.2; (b) the safe choice is determinable from captured evidence per §11.4.6 (no guessing — `LIKELY`/`probably`/`seems` is NOT a determination); (c) a wrong choice's blast radius is bounded AND recoverable; (d) it composes with anti-bluff §11.4, host-safety §12, data-safety §9. **Block-only-when (BLOCK via the §11.4.66 interactive mechanism ONLY when ALL hold):** the action is irreversible AND high-blast-radius AND the safe choice cannot be determined from evidence — e.g. external-account state the agent cannot inspect, hardware it cannot access, destructive ops without backup, force-push (also §9.2 + §11.4.41), spending money or sending data to third parties. `Operator-blocked` per §11.4.21 is reached only after this rule fires AND the self-resolution-exhaustion audit completes. An unavoidable block parks one work unit — it does NOT pause the loop; the agent keeps progressing every non-blocked item in parallel per §11.4.87 + §11.4.94 (posing the question then going idle is a §11.4.94 + §11.4.97 violation). Classification: universal (§11.4.17).

Cascade requirement: This anchor (verbatim or by `§11.4.101` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-101-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.101 for the full mandate.

§11.4.102 — Mandatory systematic-debugging activation + always-loaded skill-discovery + plugin-dependency availability (cascaded from constitution submodule §11.4.102)

> Verbatim user mandate (2026-05-29): *"Make sure that we ALWAYS trigger / start the"/superpowers:systematic-debugging" skills when any issues happen! ... we MUST activate the skill(s) and make strongest efforts in full in depth analisys / debugging and determine root causes of all problem ... we MUST make sure that "/using-superpowers" skill is ALWAYS loaded, applied and used! All dependencies (plugins) that Claude Code or other market places are offering MUST BE installed if these are not already available for loading and use!"*

Three cooperating invariants — the difference between guess-and-retry and investigate-to-root-cause-first. **(A) Mandatory systematic-debugging activation.** On ANY spotted issue / bug / test failure / gate failure / regression / misalignment / inconsistency / unexpected behaviour, the agent MUST activate `superpowers:systematic-debugging` (or the platform-equivalent structured-debugging discipline) **BEFORE proposing, writing, or applying any fix** — the **Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST.** Full four-phase arc: root-cause → pattern → hypothesis → implementation (the fix is designed only against the proven root cause). Guess-and-retry, symptom-patching, and re-running a failed test hoping it passes ("probably transient / flaky") WITHOUT a completed investigation are §11.4.102 violations; calling a failure `transient/flaky/intermittent/probably-timing` without captured forensic evidence is simultaneously a §11.4.6 (no-guessing) and §11.4.7 (demotion-evidence) violation. **(B) Mandatory always-loaded `using-superpowers`.** `superpowers:using-superpowers` (or the platform-equivalent skill-discovery / capability-index discipline) MUST be loaded and applied at session start and consulted before any task — survey available skills before acting on ANY request; if ANY skill could apply (even at 1% relevance) it MUST be invoked rather than improvised from memory. **(C) Mandatory plugin / dependency availability.** Every skill plugin / marketplace package / capability dependency the project relies on MUST be installed + loadable BEFORE the dependent work proceeds; a missing plugin that blocks a mandated skill is a release-blocker until installed + confirmed loadable (confirm by observing the skill in the live capability list — install exit 0 ≠ skill loadable, per the §11.4.80 lesson). Composes with §11.4.4 / §11.4.6 / §11.4.7 / §11.4.8 / §11.4.43 / §11.4.70 / §11.4.82(A) / §11.4.92. Classification: universal (§11.4.17). No escape hatch — no `--skip-systematic-debugging`, `--guess-and-retry-OK`, `--symptom-patch-permitted`, `--skip-skill-discovery`, `--plugin-optional`, `--missing-plugin-is-warning` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.102` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-102-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.102 for the full mandate.

§11.4.122 — No-silent-removal-of-existing-components-without-operator-confirmation mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03):

> "Never ever remove any application, system component or service from already existing codebase / System without interactively asked question to us! THIS IS MANDATORY RULE / CONSTRAINT!"

Forensic case study (FACT). During the 1.1.8-dev burn-down, two shipped capabilities — F2 (an Apple-TV-class application) and F4 (a Huawei HMS / Mobile-Services component) — were removed from the existing System WITHOUT first asking the operator; the operator reversed both. A removal the operator has to discover and reverse after the fact is a defect of the same severity class as a §11.4 PASS-bluff: the System silently lost a user-facing capability the operator never agreed to drop.

No application, system component, service, package, feature, driver, module, library, prebuilt asset — any already-existing end-user capability of the existing codebase / shipped System — may be removed (deleted, dropped from the package set, disabled-into-non-shipping, un-bundled, de-listed, or otherwise made unavailable to the end user) WITHOUT FIRST interactively asking the operator and receiving an EXPLICIT keep-or-remove decision. The question MUST be posed through the platform's interactive clarification mechanism per §11.4.66 (`AskUserQuestion` on Claude Code) — NEVER a free-text "should I remove X?" buried in narrative, NEVER a silent removal justified post-hoc, NEVER an autonomous removal decision. A silent removal is a **release blocker** regardless of how well-intentioned the rationale (deduplication, "it was broken anyway", geo-restricted, incompatible, superseded) — the operator decides, the agent asks.

What counts as a removal (non-exhaustive): deleting an app/APK/binary from the build's package set (`PRODUCT_PACKAGES` / `device.mk` / equivalent), removing a service from the init/boot/service-registry set, dropping a kernel module / driver / config from the shipping configuration, un-bundling a prebuilt asset, deleting a submodule or its shipped output, removing a feature flag that gated a live capability, or any edit whose NET EFFECT is "an end-user capability that shipped before no longer ships." Adding, replacing-with-operator-approved-equivalent, or fixing a capability is NOT a removal. When uncertain whether an edit constitutes a removal, treat it AS a removal and ask (per §11.4.6 no-guessing + §11.4.101 — removal of an existing user-facing capability is high-blast-radius and MUST be operator-confirmed, never autonomously decided). The tracked DROP path: ask → operator approves → mark the item `Obsolete (→ Fixed.md)` with `Obsolete-Details` reason `feature-removed` + an operator-approval citation (§11.4.90) → then remove; the removal never precedes the operator's yes.

Classification: universal (§11.4.17) — a platform-neutral discipline reusable by ANY project that ships a set of user-facing capabilities; the consuming project supplies its concrete capability-manifest paths per §11.4.35. Composes §11.4.66 / §11.4.101 / §11.4.90 / §11.4.112 / §11.4.6 / §11.4.40 / §11.4.42. Propagation gate `CM-COVENANT-114-122-PROPAGATION` (literal `11.4.122`) + recommended gate `CM-NO-SILENT-COMPONENT-REMOVAL` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.122. Non-compliance is a release blocker. No escape hatch — no `--remove-without-asking`, `--silent-removal`, `--autonomous-removal-OK`, `--dedup-removal-exempt`, `--it-was-broken-anyway` flag.

§11.4.123 — Rock-solid-proof-or-deep-research mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03):

> "Every single reported issue MUST BE fully and 100% validated with rock solid proofs! Nothing can be considered fixed or completed without hard evidence! No false results or bluff(s) of any kind is allowed! If we are not sure on how to achieve full testing, validation and verification of something we MUST ALWAYS perform deep web research for all possible data (articles, documentation, guides, and other resources) and opensourced

> codebases which we can use to solve our problems and perform testing with validation and verification which produces rock-solid evidence(s) and leaves no space for false results or any kind of bluff!”

Forensic case study (FACT). In the 1.1.8-dev remediation the validation method for two feature classes was, at first, genuinely unclear: relocating a `FLAG_SECURE` secure surface to a secondary display (pixel capture returns black) and asserting on-screen content in non-introspectable streaming-app UIs (blank accessibility hierarchy). Rather than declaring them “untestable” or accepting a metadata-only PASS, the cycle performed deep web research (`docs/research/testing_frameworks_20260603/`) that yielded the CV/OCR/liveness/sink-probe oracle stack (now §11.4.107 + §11.4.112 + §11.4.117) — making rock-solid evidence possible where it had appeared impossible. “Unclear how to validate” is a research trigger, NEVER a bluff licence.

Every single reported issue, every fix, and every claimed completion MUST be fully and 100% validated with rock-solid CAPTURED proof per §11.4.5 / §11.4.69 / §11.4.107 before it may be marked fixed / implemented / completed (§11.4.33 closure vocabulary). Nothing may be considered fixed or complete without hard captured evidence — metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS are all forbidden (§11.4 / §11.4.1); no false results, no bluff of any kind, at any layer.

The research-or-don’t-bluff rule (the operative addition): when the agent is UNSURE how to fully test / validate / verify something — when no obvious evidence-producing method exists OR the candidate method would yield only metadata/config/absence-of-error evidence — it MUST ALWAYS first perform deep web research per §11.4.8 + §11.4.99 (official docs, articles, guides, vendor references, standards, issue trackers, reusable open-source codebases) to DISCOVER or BUILD a validation method that produces rock-solid evidence and leaves no space for a false result. Declaring something “untestable” / “not automatable” / accepting a metadata-only PASS WITHOUT first exhausting this deep-research path is itself a §11.4.123 violation — same severity class as a PASS-bluff. The research output (cited source URLs + the evidence-producing method, OR the literal “NO external solution found — original work” per §11.4.8) is the captured proof the path was exhausted. Only after that research genuinely fails may the item be classified `PENDING_FORENSICS:` / `Operator-blocked` (§11.4.21) / `structurally-impossible` won’t-fix (§11.4.112) — with the cited research as the evidence the classification is earned, never a convenience.

Classification: universal (§11.4.17) — a platform-neutral discipline reusable by ANY project; the consuming project supplies its concrete capture mechanisms + research corpora per §11.4.35. Composes §11.4.5 / §11.4.6 / §11.4.8 / §11.4.52 / §11.4.69 / §11.4.99 / §11.4.107 / §11.4.118 / §11.4.21 / §11.4.112. Propagation gate `CM-COVENANT-114-123-PROPAGATION` (literal `11.4.123`) + recommended gate `CM-ROCK-SOLID-PROOF-OR-RESEARCH` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.123. Non-compliance is a release blocker. No escape hatch — no `--metadata-pass-suffices`, `--skip-proof`, `--untestable-without-research`, `--config-only-closure-OK`, `--bluff-when-unsure` flag.

§11.4.124 — Dead/unwired-code investigate-before-remove mandate (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

> "Before removing any seemingly-dead (zero-importer / unwired) codebase, we MUST investigate via git history where/how it was originally used and how it became dead. Removal is permitted ONLY when we have captured PROOF it is genuinely no longer needed — and that removal MUST be its own separate commit with a proper descriptive message. If there is no such proof, the code MUST be investigated for where/how it should be wired in properly, and any missing or unwired tests MUST be added. We MUST ALWAYS be extra careful with any codebase removal."

"Zero importers / never called / unwired ⇒ dead ⇒ delete" is a GUESS (§11.4.6), never a finding — a "no references" result proves only *current* non-reference, not genuinely-unneeded. Before removing ANY seemingly-dead element (zero-importer / never-called / unwired function / method / type / file / module / package / asset / config / build target) the agent MUST FIRST investigate via git history (`git log --follow`, `git log -S/-G` pickaxe across all history, blame on the deleted call-site) and capture as FACT: (1) WHERE/HOW it was originally wired in, (2) WHEN/HOW it became dead — call-site deleted deliberately / by mistake (regression) / never-completed / refactored-unreachable, (3) whether "no references" is real OR a hidden reference the static tool cannot see (reflection / dynamic dispatch / build-tags / codegen / DI / plugin registry / FFI / config-driven wiring). The investigation output (cited commits + determination) is the captured evidence. **Removal is conditional:** permitted ONLY with captured PROOF the element is genuinely no longer needed; that removal MUST be its OWN SEPARATE COMMIT (independently reviewable + revertible, composes §11.4.84 quiescence + §11.4.92 multi-pass) with a descriptive message citing the git-history evidence — plus §11.4.122 operator-confirmation when the element is an end-user capability; the §11.4.90 tracked path marks it `Obsolete (→ Fixed.md)`. **No proof ⇒ do NOT delete:** investigate WHERE/HOW to wire it in properly (restore a mistakenly-deleted call-site per §11.4.114; finish never-completed wiring) AND add any missing / unwired tests (§11.4.27 / §11.4.43 / §11.4.115 — the missing test is part of why it drifted into apparent-deadness). **Extra-caution default:** when uncertain whether removal-proof is sufficient, default to NOT removing (investigate + wire + test) per §11.4.6 + §11.4.101 + §11.4.122; "probably dead" is never sufficient — the bar is captured proof. Classification: universal (§11.4.17) — the consuming project supplies its static-analysis / importer-graph tooling + hidden-reference mechanisms per §11.4.35. Composes §11.4.6 / §11.4.8 / §11.4.84 / §11.4.90 / §11.4.92 / §11.4.101 / §11.4.114 / §11.4.122 / §11.4.27 / §11.4.43 / §11.4.115. Propagation gate `CM-COVENANT-114-124-PROPAGATION` (literal `11.4.124`) + recommended gate `CM-DEAD-CODE-INVESTIGATE-BEFORE-REMOVE` (a net-deletion commit must be removal-only + cite the git-history investigation OR be part of a tracked Obsolete item) + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.124. Non-compliance is a release blocker. No escape hatch — no `--zero-importers-means-dead`, `--delete-unwired-on-sight`, `--skip-git-history-investigation`, `--remove-without-proof`, `--bundle-removal-with-other-work` flag.

§11.4.125 — Code-review-agent gate before pre-build + main build (mandatory multi-layer review) (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

> "After all fixes/changes/implementations are done, BEFORE running pre-build tests and the main build, dispatch code-review agent(s) that analyze all work done + all existing data/facts + the existing codebase + current git history to determine quality, safety, and whether the fixes/changes will REALLY work; they MUST validate and verify that every test covering the fixes/changes genuinely validates the work with NO chance of false results or bluff of any kind. Any finding MUST be fixed, polished, improved, and covered with additional tests before the build proceeds. Multiple strong layers of checks."

After all fixes / changes / implementations in a batch are done, and BEFORE running the pre-build test sweep AND the main (artifact) build (for ANY project), the agent MUST dispatch one or more dedicated code-review agent(s) (subagent-driven by default per §11.4.70/§11.4.20) performing a multi-layer review that: (1) analyzes ALL work done in the batch (every fix/change + its source diff + stated intent); (2) analyzes ALL existing data + facts (captured evidence per §11.4.5/§11.4.69/§11.4.107, tracker entries, prior findings, the §11.4.108 runtime-signature registry); (3) analyzes the existing codebase (blast radius per §11.4.92, cross-feature interaction, contract integrity of every dependency); (4) analyzes current git history (what each change touched, how it composes with concurrent/recent work, whether it reproduces a known-broken pattern per §11.4.114/§11.4.124); (5) determines quality + safety + will-it-REALLY-work (robust + not error-prone — no solve-A-create-B; no host/data/security regression; genuinely delivers the end-user-visible behaviour per §11.4/§107); (6) validates + verifies the tests covering the work — every covering test genuinely exercises the work-under-test and catches its negation, with ZERO chance of a false result or bluff (a test that PASSes on broken-for-the-user work, a metadata-only/config-only/absence-of-error/grep-without-runtime assertion, or a gate whose paired §1.1 mutation does not make it FAIL is a finding). Any finding (defect / error-prone change / safety risk / will-not-really-work / bluff-or-false-result-capable test / missing-coverage gap) MUST be fixed, polished, improved, and covered with additional tests (four-layer per §11.4.4(b), TDD-RED-first per §11.4.43/§11.4.115) BEFORE the pre-build sweep + main build proceed; the review iterates (re-review after each remediation) until no blocking findings remain. The review is itself anti-bluff (its conclusions are captured evidence per §11.4.5/§11.4.69; a rubber-stamp review of a defective batch = PASS-bluff). It is one of MULTIPLE STRONG LAYERS — complementing, never replacing, the §1 pre-build sweep, §11.4.92 multi-pass (author-side self-review; §11.4.125 adds the structurally-separated reviewer seam per §11.4.70), §11.4.108 four-layer fix-verification, §11.4.110 build-readiness verdict, and the post-build / runtime-on-clean-target / user-visible layers. Composes §11.4 / §11.4.1 / §11.4.4 / §11.4.6 / §11.4.40 / §11.4.43 / §11.4.50 / §11.4.70 / §11.4.20 / §11.4.92 / §11.4.102 / §11.4.107 / §11.4.108 / §11.4.110. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-125-PROPAGATION` (literal `11.4.125`) + recommended gate `CM-CODE-REVIEW-GATE-BEFORE-BUILD` (build starts only with a fresh code-review-completed marker for the current batch, produced after the last fix + before the pre-build sweep + main build) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.125. Non-compliance is a release blocker. No escape hatch — no `--skip-code-review`, `--build-without-review`, `--no-review-gate`, `--review-optional`, `--trust-the-author` flag.

§11.4.126 — Default autonomous-loop working mode from first prompt (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04):

> "Make sure that you continue work in endless fully autonomous loop, do not stop until new fully validated and verified version (tag) is created and published (all submodules and main repo) or IN A CASE OF some other main stream work until it is fully completed with all side work streams and nothing else is left in our working queue! THIS MUST BE ALWAYS the default working mode without us asking you! We tend to achieve ABSOLUTE EFFICIENCY, with this and all other projects which will incorporate this MANDATORY RULE / CONSTRAINT!!! This way of (your) working will be ALWAYS applied / followed / executed / fully respected, as soon as we assign / send first request (prompt) in the session! This stops only if we explicitly say so or nothing is left to be done in current working scope (release that will come / upcoming version)!!! Any mimicking (imitation) of this behavior / rules / mandatory constraints, false results or any kind of bluff(s) is ABSOLUTELY FORBIDDEN!!!"

The endless fully-autonomous loop is the **DEFAULT working mode**, engaged automatically the moment the operator sends the FIRST request / prompt of a session — the operator MUST NOT have to ask for it, request it, restate it, or re-enable it per session. §11.4.87 framed the endless-loop covenant as an explicit-instruction opt-in ("continue in endless loop fully autonomously" or a semantically-equivalent phrasing); §11.4.126 is the **capstone** that promotes the same covenant to always-on: from the first prompt onward, every agent operates in the §11.4.87 loop discipline as the standing default, with §11.4.94 zero-idle, §11.4.97 maximum-idle-use, §11.4.101 autonomous-decision-over-blocking, and §11.4.103 continuous-parallel-stream all engaged by default — no per-session activation handshake. The continuation contract: the loop continues until ONE of two terminal conditions holds — (A) **Release scope** — a new, fully-validated-and-verified version (tag) is created AND published across all owned submodules AND the main repo to all configured remotes (per §2.1 multi-upstream push + §11.4.40 full-suite-retest-before-tag + §11.4.113 absolute-no-force-push merge-onto-latest-main); OR (B) **Non-release main-stream scope** — the main-stream goal is fully completed AND every side work stream is done AND the working queue holds nothing left for the current scope. Until (A) or (B) holds, the agent MUST keep working (claim the next priority item, dispatch the next parallel stream, progress every non-blocked item per §11.4.42 / §11.4.72 / §11.4.94 / §11.4.103). The loop STOPS ONLY on: (1) the operator explicitly saying so (STOP / pause / end); (2) nothing left to do in the current working scope — the upcoming release / current main-stream goal — with the queue genuinely empty per the (A)/(B) terminal conditions; (3) a §12 host-session-safety demand (the loop yields to host safety unconditionally). Idle-while-blocked parks one work unit, it does not stop the loop — the agent keeps progressing every non-blocked item in parallel per §11.4.101 + §11.4.94 + §11.4.97. Goal — ABSOLUTE EFFICIENCY (no operator-side restart overhead, no idle gaps, no stop-and-wait round-trips); applies to this project AND every project that incorporates this Constitution. Anti-bluff: mimicking / imitating this loop behaviour,

narrating continuation without performing it, fabricating progress, or emitting false / bluff results of ANY kind is ABSOLUTELY FORBIDDEN — this composes the entire §11.4 anti-bluff covenant family (§11.4 / §11.4.1 / §11.4.2 / §11.4.5 / §11.4.6 / §11.4.50 / §11.4.69 / §11.4.107); the agent MUST genuinely perform the continuous work and capture positive evidence for every closure, and a report claiming the loop ran while no real work / no captured evidence was produced is a §11.4 PASS-bluff at the operating-mode layer. Classification: universal (§11.4.17). Composes with §11.4.87 (the endless-loop covenant — §11.4.126 promotes it from opt-in to always-on default) / §11.4.94 / §11.4.97 / §11.4.101 / §11.4.103 / §11.4.66 / §11.4.6 / §11.4.40 / §11.4.42 / §11.4.72 / §11.4.113 / §2.1 / §12. Propagation gate `CM-COVENANT-114-126-PROPAGATION` (literal `11.4.126` across the consumer fleet) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.126. Non-compliance is a release blocker. No escape hatch — no `--ask-before-continuing`, `--single-turn-only`, `--not-default-loop`, `--mimic-OK` flag.

§11.4.127 — Session-handoff resumption-prompt mandate (User mandate, 2026-06-06)

Forensic anchor — verbatim user mandate (2026-06-06): “make sure that in situations like this now when new session is needed you ALWAYS prepera such sentence - which will be valid for particular moment and the phase of the project and enough for work to continue.”

When the agent determines a fresh session is needed (context-window limits, performance degradation) OR the operator asks whether a new session is needed / requests a handoff, the agent MUST ALWAYS prepare + proactively provide a ready-to-paste **resumption prompt valid for that EXACT moment and project phase** — self-contained enough that pasting it into a fresh session resumes work with ZERO loss. Two variants on demand: a SHORT first-sentence (“Read `<handoff docs>`, then continue `<terminal goal>` …”) AND a FULL detailed block. The prompt MUST: (1) point to the live handoff doc(s) — `.remember/remember.md` if present + `docs/CONTINUATION.md` per §12.10 — read FIRST + `git fetch --all`; (2) state current PHASE + immediate NEXT action + terminal goal; (3) embed exact live-state anchors (build IDs / artifact MD5, device/target serials, commit HEAD, in-flight PIDs + log paths, captured-evidence paths); (4) restate binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware/target gotchas); (5) be MOMENT-VALID, NEVER a generic template. Handoff doc(s) MUST be current BEFORE the prompt is given (§12.10). A missing / stale / generic prompt is a §11.4.127 violation. Composes §12.10 / §11.4.6 / §11.4.66 / §11.4.87 / §11.4.103 / §11.4.126. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-127-PROPAGATION` (literal `11.4.127`) + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.127. Non-compliance is a release blocker. No escape hatch — no `--skip-handoff-prompt`, `--generic-prompt-OK`, `--no-resumption-sentence`, `--handoff-without-state` flag.

§11.4.128 — Always-on device-recording mandate (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): we MUST ALWAYS live-record all available data from all devices we use for testing (or known to be under manual testing), EXTRA carefully so it never harms the device / its performance / causes side effects; raw recordings are NOT processed without need (token-conscious) and are ALWAYS git-ignored + code-intelligence-excluded; only curated evidence is committed, and only at release prep.

For EVERY test/debug device the project uses + every device under known manual testing, across EVERY reachable transport (USB / wireless ADB / SSH / serial / network introspection API), the project MUST ALWAYS live-record all analysable data: activities, all logs, performance metrics (CPU/memory/I/O/thermal/load), every sink-side report per §11.4.13, and any other live-changeable parameter. (1) **Extra-careful, side-effect-free** — non-invasive read-only probes only, bounded sampling, bounded write-volume, an observer-effect budget; a recorder that perturbs the device-under-test is a §11.4.128 violation, NOT evidence. (2) **Background + parallel + subagent-driven** per §11.4.103 + §11.4.70 — never blocks the main stream. (3) **Token-conscious — record-now, analyse-later** — raw data NOT processed without need; the only standing analyse-trigger is release-tag prep (§11.4.40 / §11.4.42) OR explicit operator ask. (4) **Raw is git-ignored (with a §11.4.77 regen-mechanism declaration) AND code-intelligence-excluded (§11.4.78/§11.4.79)** — only CURATED evidence is committed, and only at release prep under `docs/qa/<run-id>/` (§11.4.83). (5) **Deterministic layout** `<recording-root>/YYYY-MM-DD/<combined main+submodules state hash>/<DEVICE>_<SERIAL>/recording_NNN/<files>`. (6) **Anti-bluff** — a recorder claimed running but with no growing corpus is a §11.4 bluff; every curated finding traces to a real raw-corpus path; recorder health is itself captured evidence per §11.4.5/§11.4.69.

Composes §11.4.2 / §11.4.5 / §11.4.13 / §11.4.69 / §11.4.40 / §11.4.42 / §11.4.70 / §11.4.77 / §11.4.78 / §11.4.79 / §11.4.83 / §11.4.103 / §11.4.119. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-128-PROPAGATION` (literal `11.4.128`) + recommended gate `CM-DEVICE-RECORDING-ALWAYS-ON` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.128. Non-compliance is a release blocker. No escape hatch — no `--skip-recording`, `--record-without-layout`, `--commit-raw-corpus`, `--index-raw-corpus`, `--analyse-corpus-always`, `--invasive-probe-OK` flag.

§11.4.129 — Huge-blocker release protocol (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): when a huge blocker is discovered during release validation we MUST stop all testing, fix ALL discovered issues, process all recorded data from the last session, land rock-solid fixes, author NEW validation+verification tests of ALL supported test types, rebuild, reflash, and RESTART the full validation+verification of every fix/change from the last release tag to now — on both devices in parallel, recorded, with real physical captured proofs and no bluff.

On discovery of a HUGE BLOCKER (release-blocking-severity defect: core user-facing capability broken, regression invalidating the in-flight cycle, or blast radius reaching the batch's other fixes) during release validation, execute in order with NO spot-check shortcut: (1) **STOP all testing** on every device (the §11.4.4 test-interrupt STOP at release granularity — continuing past a huge blocker is the §11.4 PASS-bluff). (2) **Fix ALL discovered issues** — not just the blocker; root-cause each per §11.4.102 + isolate regressions against the last known-good tag per §11.4.114. (3) **Process all recorded data from the last session** — analyse the §11.4.128 raw-corpus slice (this IS the §11.4.128(3) release-prep analyse-trigger). (4) **Land rock-solid fixes** per §11.4.123 + §11.4.43/§11.4.115 + §11.4.9. (5) **Author NEW validation+verification tests of ALL supported test types** per §11.4.27 + §11.4.85, each anti-bluff + paired §1.1 mutation. (6) **Rebuild (full, not module-only) + reflash to a CLEAN target** per §11.4.108. (7) **RESTART the full validation+verification from the last release tag to now** per §11.4.40 — RESTART, never resume — on both/all owned devices IN PARALLEL per §11.4.103/§11.4.119, every run RECORDED per §11.4.128, real physical captured proofs per §11.4.5/§11.4.69/§11.4.107, no bluff. This anchor BINDS the existing release anchors for the huge-blocker case (adds STOP→fix-all→process-recordings→new-tests-all-types→rebuild→reflash→full-restart + the restart-not-resume rule), citing them rather than duplicating.

Composes §11.4.4 / §11.4.40 / §11.4.42 / §11.4.9 / §11.4.27 / §11.4.85 / §11.4.102 / §11.4.108 / §11.4.114 / §11.4.115 / §11.4.123 / §11.4.128 / §11.4.103 / §11.4.119. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-129-PROPAGATION` (literal `11.4.129`) + recommended gate `CM-HUGE-BLOCKER-FULL-RESTART` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.129. Non-compliance is a release blocker. No escape hatch — no `--resume-after-blocker`, `--spot-validate-after-fix`, `--skip-recording-analysis`, `--skip-new-tests`, `--module-only-after-blocker`, `--single-device-restart` flag.

§11.4.130 — Post-remediation validate-the-fix-FIRST-after-redeploy (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): when a blocker discovered during release validation is fixed and a new artifact (rebuild / new flashing image / redeploy) is produced + the target reflashed, we MUST first re-test the SPECIFIC last-failing features + validate the just-incorporated fixes BEFORE the broader / full validation.

When a blocker / critical failure found during release validation is FIXED and a new artifact is produced + the target reflashed / redistributed / updated, the agent MUST: (1) **re-test the SPECIFIC last-failing features FIRST** (targeted guard tests for exactly the defects this fix addressed) BEFORE any broader / full-suite validation; (2) **validate the just-incorporated fixes with real captured evidence** — the §11.4.115 RED test flips GREEN at `RED_MODE=0` on the new artifact AND the §11.4.108 runtime-signature verifies on the CLEAN target the redeploy produced (metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden per §11.4 / §11.4.1; proof per §11.4.5/§11.4.69/§11.4.107/§11.4.123); (3) **only after the targeted fix is CONFIRMED working** proceed to the §11.4.40 full retest from the last tag to now. Rationale: a first fix attempt may not work / may be incomplete / may regress again under the new artifact — confirming the targeted fix FIRST catches a fix-did-not-take case immediately instead of hours later at the END

of a full cycle (then restarting per §11.4.129); cheap-confirmation-first is §11.4.82 applied to the post-blocker reflash. This is the §11.4.46 recent-work-validation gate specialised for the post-blocker-reflash case + the targeted-confirmation phase that GATES §11.4.129's step-7 full-restart. Honest boundary (§11.4.6): "the fix probably took" ≠ "the fix took" — the RED→GREEN flip + runtime-signature on the new artifact is the proof; a still-FAILing targeted re-test re-enters the §11.4.114/§11.4.115 isolate→RED→fix loop, never proceeds to the full cycle on a still-broken fix. Composes §11.4.4 / §11.4.40 / §11.4.46 / §11.4.108 / §11.4.114 / §11.4.115 / §11.4.123 / §11.4.129 / §11.4.82. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-130-PROPAGATION` (literal `11.4.130`) + recommended gate `CM-FIX-FIRST-AFTER-REDEPLOY` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.130. Non-compliance is a release blocker. No escape hatch — no `--skip-targeted-retest`, `--full-cycle-first`, `--assume-fix-took`, `--validate-fix-at-end`, `--skip-red-green-flip-on-new-artifact` flag.

§11.4.131 — Standing session-resumption file mandate (User mandate, 2026-06-07)

Forensic anchor — verbatim user mandate (2026-06-07): "Make this markdown a standard file which will be written EVERY TIME when we need fresh session out of the box! It MUST BE always up to date and in sync so whenever new session is created all we have to do is just point to it!"

Every project MUST maintain a SINGLE canonical, always-current **session-resumption file** at a fixed, project-declared standard path (declared once per §11.4.35, never moved without a §11.4.66 operator decision). This file is the OUT-OF-THE-BOX entry point for any fresh session: creating a new session requires ONLY pointing the new agent at this one file. §11.4.131 promotes §11.4.127 (PREPARE a resumption prompt on demand) into a STANDING, version-controlled ARTIFACT — ALWAYS present, ALWAYS in sync. (A) **Existence + fixed path** — exists at the declared path at all times, encoded as a literal path in the project-layer instantiation (§11.4.35), never silently moved. (B) **Always written + always synced** — (re)written whenever a fresh session is needed OR the live state materially changes (new HEAD, build/artifact id, phase, device/target state, in-flight job, blocking decision) — the §12.10 trigger set; a stale resumption file is a §11.4.131 violation of the same severity class as a §12.10 stale-CONTINUATION violation. (C) **Content (composes §11.4.127)** — both SHORT + FULL variants; points to `.remember/remember.md` + `docs/CONTINUATION.md` read FIRST + `git fetch`; embeds exact live-state anchors (HEAD, build/artifact ids + checksums, device serials, in-flight PIDs + log paths, captured-evidence paths); states PHASE + immediate NEXT + terminal goal; restates binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware gotchas); MOMENT-VALID, never a generic template (§11.4.6). (D) **Export + freshness** — §11.4.65 scope (synchronized `.html`/`.pdf` siblings) + §11.4.44 revision header. (E) **Out-of-the-box resumption** — a fresh session, given ONLY this file's path, fully resumes with zero additional context. Composes §12.10 / §11.4.127 / §11.4.65 / §11.4.44 / §11.4.6 / §11.4.66 / §11.4.126. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-131-PROPAGATION` (literal `11.4.131`) + recommended gate `CM-SESSION-RESUMPTION-FILE-PRESENT` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.131. Non-compliance is a release blocker. No escape hatch — no `--skip-resumption-file`, `--ephemeral-prompt-only`, `--stale-resumption-OK`, `--generic-template-OK` flag.

§11.4.132 — Risk-ordered validation priority mandate (User mandate, 2026-06-07)

Forensic anchor — verbatim user mandate (2026-06-07): "We MUST ALWAYS first test and validate features, functionalities and fixes/changes that have been worked most recently, the ones which were most problematic, which have the most chance to crash or break again, the ones which have been re-opened the most times! Then, after we validate and verify all this with real (physical) proofs and hard evidence, with no false results and bluffs of any kind, we continue with all other existing tests in the test suites! This IS MANDATORY."

Tests / validations / verifications MUST run in **RISK-DESCENDING order** — the highest-risk set FIRST, and ONLY AFTER that set is fully GREEN with real (physical) captured evidence does the remainder of the suite run. Risk ranking is computed from a CLOSED set of factors, highest-risk first: (a) **most-recently-worked** features / fixes / changes; (b) **historically most-problematic** (longest defect history, most prior fixes/failures); (c) **highest crash/break/regress likelihood** (greatest blast radius / complexity / dependency surface); (d) **most-reopened** per §11.4.55 reopens-count (a high reopen count is the strongest empirical fragility signal). Each item in the highest-risk set MUST pass with real (physical) captured evidence per §11.4.5/§11.4.69/§11.4.107 — no metadata-only / config-only / absence-of-error / grep-without-runtime PASS (§11.4/§11.4.1), no false results, no bluff (§11.4.6). ONLY AFTER the entire highest-risk set is GREEN with captured proof does the rest of the suite run; running the suite in arbitrary order, or running lower-risk tests before the highest-risk set is GREEN, is a §11.4.132 violation. §11.4.132 REFINES/STRENGTHENS §11.4.130 (generalises "validate the just-fixed items first" to the full risk-ordered set) + §11.4.46 (adds explicit risk-ordering within the recent/high-risk set) + §11.4.42 (applies the implementation-layer priority discipline to VALIDATION ordering). Classification: universal (§11.4.17) — the consuming project supplies its recency / problematic-history / reopen-count sources (e.g. §11.4.93 workable-items DB `reopens_count`+`last_modified`) per §11.4.35. Composes §11.4.4/.5/.6/.7/.40/.42/.46/.50/.55/.69/.107/.130. Propagation gate `CM-COVENANT-114-132-PROPAGATION` (literal `11.4.132`) + recommended gate `CM-RISK-ORDERED-VALIDATION-PRIORITY` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.132. Non-compliance is a release blocker. No escape hatch — no `--skip-risk-ordering`, `--any-order-OK`, `--suite-order-fixed` flag.

§11.4.133 — Target-System + hardware safety mandate (User mandate, 2026-06-08)

Forensic anchor — verbatim user mandate (2026-06-08): "Make sure that all changes we do to the System are ALWAYS safe for the System itself and for the hardware the system runs on! This is MANDATORY."

Every change to the TARGET system (firmware, kernel, init/boot scripts, drivers, sysfs/devfreq/voltage/clock/thermal/regulator register writes, partition/bootloader/U-Boot, HAL, framework, prebuilts, device config) MUST ALWAYS be safe for BOTH (a) the target System itself — MUST NOT brick, boot-loop, corrupt data, or render the device unrecoverable — AND (b) the hardware it runs on — MUST NOT exceed safe electrical/thermal/voltage/clock limits or damage panels/storage/radios/regulators. Concrete obligations: (1) reversible-first — verify irreversible high-blast-radius changes (bootloader/U-Boot MD5, partition layout) against known-good values + capture a pre-op backup (§9.2) BEFORE applying; (2) NO unverified hardware-control writes — never write an unverified value to a voltage/clock/regulator/thermal-throttle/current-limit sysfs node or register that could exceed datasheet limits, the safe range established as FACT (§11.4.6), never guessed; (3) thermal/perf changes (forcing a performance governor, pinning the top OPP, disabling thermal management) MUST respect the device's cooling design, validated by captured thermal evidence; (4) flashing MUST use the sanctioned tool + a freshly-built integrity-verified image — never an ad-hoc partition write or stale/unverified artifact; (5) unprovable-safety ⇒ blocked — a change whose target/hardware safety cannot be established from captured evidence is treated as UNSAFE and blocked (§11.4.6 + §11.4.101 reversible-first + §11.4.123 rock-solid-proof). DISTINCT from §12 host-session safety: §12 protects the DEVELOPER's HOST + session; §11.4.133 protects the TARGET device + its hardware — both apply, neither weakens the other. Classification: universal (§11.4.17) — the consuming project supplies its concrete hardware-control surfaces, datasheet-safe ranges, known-good bootloader/image hashes, and sanctioned flashing tool per §11.4.35. Composes §12 / §11.4.6 / §11.4.101 / §11.4.108 / §11.4.123. Propagation gate `CM-COVENANT-114-133-PROPAGATION` (literal `11.4.133`) + recommended gate `CM-TARGET-HARDWARE-SAFETY` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.133. Non-compliance is a release blocker. No escape hatch — no `--unsafe-hardware-write`, `--skip-system-safety`, `--brick-risk-accepted` flag.

§11.4.134 — Code-review iterate-until-GO + rock-solid-evidence mandate (User mandate, 2026-06-08)

Forensic anchor — verbatim user mandate (2026-06-08): "For any fixes/changes given back to us for re-work by the code-review process, once we fix/improve everything per the code-review's requests, we MUST RE-RUN code-review AGAIN until we get a GO from it with NO new issues reported or warnings of any kind! All results produced by this whole process MUST ALWAYS give us rock-solid PHYSICAL evidence that the fixed/improved codebase really works now as expected, with no false results and no bluff(s) of any kind."

When the §11.4.125 code-review returns ANY finding — BLOCKING, nit, or warning — and the author fixes/improves the batch per that review, the code review MUST BE RE-RUN, and MUST KEEP being re-run after each remediation round, until it returns a clean GO with ZERO new issues AND ZERO warnings of any kind. A single pass that "addressed the findings" is NOT sufficient: the corrected batch MUST pass a FRESH adversarial review (a re-review can surface NEW findings introduced by the very fixes that closed the prior ones — the §11.4.1 fix-A-creates-B failure mode). The loop terminates ONLY on a clean GO (no new findings, no warnings); a residual warning is itself a finding that re-arms the loop. Every round's verdict AND every fix's validation MUST carry rock-solid PHYSICAL captured evidence per §11.4.5 / §11.4.69 / §11.4.107 (captured audio /

video / sysfs / dumpsys / sink-side / runtime-signature) proving the fixed/improved codebase REALLY works as expected — never metadata-only / configuration-only / absence-of-error / grep-without-runtime; no false results, no bluff at any round; a reported GO unbacked by captured physical evidence is itself a §11.4 PASS-bluff at the review-loop layer. §11.4.134 REFINES / STRENGTHENS §11.4.125 (iterate "until no blocking findings remain"): it makes the loop EXPLICIT (re-run after every remediation round, not once), raises termination to ZERO findings AND ZERO warnings (not merely zero-blocking), and BINDS rock-solid physical evidence to every round. Classification: universal (§11.4.17). Composes §11.4.125 / §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.69 / §11.4.107 / §11.4.50 / §11.4.108 / §11.4.123. Propagation gate `CM-COVENANT-114-134-PROPAGATION` (literal `11.4.134`) + recommended gate `CM-CODE-REVIEW-ITERATE-UNTIL-GO` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.134. Non-compliance is a release blocker. No escape hatch — no `--skip-rereview`, `--single-review-pass`, `--warnings-ok`, `--evidence-optional` flag.

§11.4.135 — Standing regression-guard suite + every-fixed-defect-gets-a-permanent-regression-test (User mandate, 2026-06-08). Every project MUST maintain a STANDING regression-guard suite that runs on EVERY build+deploy and BLOCKS the release tag on any failure. Every closed defect (stable ticket id, e.g. ATM-NNN) MUST, in the SAME commit as its fix (extending the §11.4.43 DOCUMENT step), register a permanent §11.4.115 RED-on-broken-artifact regression test into the suite — `RED_MODE=1` capturing the historical defect on a pre-fix artifact (the proof the guard is real), `RED_MODE=0` the standing GREEN guard asserting the defect is ABSENT. A closure without a registered guard is a §11.4.123 violation. The suite runs FIRST in the post-deploy cycle (highest-risk set per §11.4.132) and is a §11.4.40 release-gate blocker. Forensic anchor (FACT): the wrong-subtitle-on-2nd-display defect was "fixed" via a source-side `CONTROL_MENU_LABEL_DENYLIST` that NO test mirrored or re-ran, so the NEXT chrome class recurred silently while the GREEN suite passed. Industry-standard bug-driven testing (Google content-driven testing; AOSP CTS/Tradefed) made mechanical + enforced. Composes §11.4.4 / §11.4.40 / §11.4.43 / §11.4.46 / §11.4.50 / §11.4.107 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.124 / §11.4.130 / §11.4.132. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-135-PROPAGATION` (literal `11.4.135`) + recommended gates `CM-REGRESSION-GUARD-REGISTERED` / `CM-REGRESSION-GUARD-SUITE-WIRED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.135. Non-compliance is a release blocker. No escape hatch — no `--skip-regression-guard`, `--no-guard-on-close`, `--guard-optional` flag.

§11.4.136 — Real-content end-to-end playback-test mandate (User mandate, 2026-06-08). Refines/strengthens §11.4.107. Any test asserting media playback works MUST drive REAL content (catalog stream or offline reference clip) through the user's path (§11.4.48 UI-driven → §11.4.117 CV/OCR fallback) and assert it genuinely PLAYS via the §11.4.107 liveness battery PLUS a decoder-health census — a numeric drop-buffer budget, no buffer-timestamp re-order/discard, no codec-reject (cite Android/Media3 ExoPlayer OEM pre-OTA playback-test mandate: "too many dropped buffers" >25, "unexpected presentation timestamp", "test timed out"). Metadata-only / launch-only / registration-only / single-frame / config-only PASS is forbidden (§11.4 / §11.4.1). A golden/reference clip corpus (BBC ExoPlayer testing samples) is the offline ground-truth. Composes §11.4.5 / §11.4.48 / §11.4.50 / §11.4.107 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Classification: universal

(§11.4.17). Propagation gate `CM-COVENANT-114-136-PROPAGATION` (literal `11.4.136`) + recommended gate `CM-REAL-CONTENT-PLAYBACK-TEST` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.136. Non-compliance is a release blocker. No escape hatch — no `--launch-proves-playback`, `--skip-decoder-health`, `--metadata-playback-pass-suffices` flag.

§11.4.137 — Subtitle/caption content-correctness oracle + secure-display-proxy-honesty mandate (User mandate, 2026-06-08). Refines §11.4.117 + §11.4.107 + §11.4.112. Forensic anchor (FACT): tests tasked to "physically verify the 2nd-display subtitle" PASSed GREEN while subtitles did NOT show / showed WRONG — the streaming player surface is FLAG_SECURE so `screencap -d <secondary>` returns BLACK (autonomous PIXEL verification structurally impossible per §11.4.112), so the test fell back to the accessibility-scraped/ `persist.atmosphere.subdebug` proxy, and the proxy accepted a chrome/menu LABEL (`Аудио и субтитры`) as a valid subtitle because the prose floor accepted any multibyte prose and NO menu-label denylist + NO position/cadence check existed. The mandate: a subtitle-correctness test MUST classify the cue's *content class* — a present cue is NOT a correct cue. CHROME (FAIL) if a known control/ menu label (closed multilingual deny-list MIRRORED from source, case-folded incl. non-ASCII), time/numeric chrome, not prose, OUTSIDE the lower safe-title band (CEA-708 9-anchor grid), OR STATIC across the window (real subtitle changes → ≥2 distinct prose cues, a metamorphic relation). DIALOGUE (PASS) only when prose + not-denied + not-chrome + position-ok + cadence ≥2 OR fuzzy-matches the SOURCE-extracted cue via normalized edit distance (§11.4.123 host ground truth). The oracle MUST be self-validated golden-good/golden-bad (§11.4.107(10)) and the deny-list MUST be verified present in the SHIPPED artifact (§11.4.108) — a source-green denylist with no test mirror + no artifact check is the exact recurrence pattern forbidden here. Secure-display honesty (§11.4.112): where FLAG_SECURE makes pixel verification impossible, the rock-solid autonomous proof is the player's caption telemetry + source-track presence + content-class oracle — NEVER a faked pixel "physical" pass; human-eye pixel confirmation is `operator_attended` (§11.4.52) with a tracked migration item. App-agnostic (keys off content class). Composes §11.4.3 / §11.4.5 / §11.4.6 / §11.4.107 / §11.4.108 / §11.4.112 / §11.4.115 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-137-PROPAGATION` (literal `11.4.137`) + recommended gate `CM-SUBTITLE-CONTENT-CORRECTNESS-ORACLE` + paired §1.1 mutation (strip the denylist/position/cadence check → golden-bad `Аудио и субтитры` PASSes → gate FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.137. Non-compliance is a release blocker. No escape hatch — no `--present-cue-is-correct`, `--skip-chrome-oracle`, `--length-heuristic-suffices`, `--pixel-pass-on-secure-display`, `--skip-position-check`, `--skip-cadence-check` flag.

§11.4.138 — Operator-escape => mandatory bluff-audit + permanent guard (User mandate, 2026-06-08). When the operator (or any out-of-band channel) finds a defect that the GREEN test suite passed, this is by definition a §11.4 PASS-bluff — it MUST trigger, before the fix is closed: (1) a §11.4.102 systematic-debugging pass to FACT-root-cause; (2) a bluff-audit identifying the EXACT assertion that should have caught it but didn't, cited to `file:line` (canonical example: `lib/subtitle_content_validation.sh:sub_is_prose()` returning TRUE for `Аудио и субтитры`); (3) a permanent §11.4.135 regression guard registered in the SAME commit as the fix, with its §11.4.115 RED capturing the operator-found defect; (4) the bluff-audit committed under `docs/research/<scope>/<defect>_bluff_audit/`. Closing an operator-found defect WITHOUT the bluff-

audit + permanent guard is itself a §11.4 violation (the bluff that let it through is still live and the defect will recur). Composes §11.4 / §11.4.1 / §11.4.102 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.135. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-138-PROPAGATION` (literal `11.4.138`) + recommended gate `CM-OPERATOR-ESCAPE-BLUFF-AUDIT` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.138. Non-compliance is a release blocker. No escape hatch — no `--close-without-bluff-audit`, `--operator-find-is-just-a-bug`, `--skip-permanent-guard` flag.

§11.4.139 — Fresh-process clean-artifact runtime-signature mandate (User mandate, 2026-06-08). Refines §11.4.108. Before any post-deploy validation — ESPECIALLY a non-pixel proxy verification (the subdebug/accessibility-cue channel used for FLAG_SECURE displays) — the harness MUST assert running-artifact == built-artifact: the deploy yielded a CLEAN target (mutable-overlay/ userdata wiped) OR a pre-validation check proves no stale overlay shadows the deployed code (e.g. every guarded package — incl. the Presenter that emits the subtitle cue — resolves to the system partition, no per-user override). A stale shadow of the cue-emitting component (e.g. a Presenter APK predating the denylist) makes the proxy report on code that was never deployed — any PASS is a §11.4 PASS-bluff. Each fix declares ONE machine-checkable runtime signature verified on the clean target (the §11.4.108 registry IS the definition of done); for the subtitle class the signature is "the shipped Presenter APK contains the denylist literal (case-insensitive) AND the subdebug channel emits `candidate REJECTED reason=chrome-label` for a menu label." Composes §11.4.46 / §11.4.108 / §11.4.130 / §11.4.135 / §11.4.137. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-139-PROPAGATION` (literal `11.4.139`) + recommended gate `CM-CLEAN-ARTIFACT-RUNTIME-SIGNATURE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.139. Non-compliance is a release blocker. No escape hatch — no `--validate-against-running-state`, `--skip-clean-precondition`, `--shadow-OK` flag.